INSTITUTO TECNOLÓGICO DE TIJUANA

SEMESTRE:
SEXTO SEMESTRE (ENERO - AGOSTO 2026)

CARRERA:
INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:
Lenguaje de Interfaces

NOMBRE Y NÚMERO DE CONTROL DE LOS ALUMNOS:
Medina Lopez Emilio Ignacio - 23212010

NOMBRE DEL MAESTRO:
Rene Solis Reyes

FECHA DE ENTREGA:
23 de abril del 2026

Documentación técnica – Mini Cloud Log Analyzer

1. Descripción general

El proyecto **Mini Cloud Log Analyzer** es una práctica desarrollada en **ensamblador ARM64**, cuyo objetivo es analizar un archivo de registros que contiene códigos HTTP, clasificándolos en categorías y mostrando estadísticas finales.

El programa lee desde la entrada estándar una lista de códigos como:

```
200
201
404
500
503
```

Y genera una salida con:

- Total de códigos leídos
- Cantidad de códigos exitosos **2xx**
- Cantidad de errores del cliente **4xx**
- Cantidad de errores del servidor **5xx**

La salida esperada es:

```
=== Mini Cloud Log Analyzer ===
------------------------------
Total de codigos: 5
Exitos 2xx: 2
```

Errores 4xx: 1
Errores 5xx: 2

2. Objetivo de la práctica

La práctica busca reforzar los siguientes conceptos:

- Uso de **syscalls** en ARM64
- Manejo de **buffers**
- Conversión de caracteres ASCII a enteros
- Clasificación mediante comparaciones numéricas
- Escritura a salida estándar
- Automatización con `Makefile`

3. Estructura del proyecto

El proyecto está organizado de la siguiente manera:

```
mini-cloud-log-analyzer/
│── src/
│   └── analyzer.s
│── data/
│   └── logs_A.txt
│── Makefile
│── run.sh
```

Descripción de archivos:

`src/analyzer.s`

Contiene la lógica principal en ensamblador ARM64:

- Lee el archivo desde `stdin`
- Procesa línea por línea
- Clasifica los códigos
- Imprime resultados

`data/logs_A.txt`

Archivo de entrada con códigos HTTP de prueba.

Ejemplo:

```
200
201
204
404
418
500
503
```

`Makefile`

Automatiza la compilación del proyecto usando `as` y `ld`.

`run.sh`

Script que ejecuta el analizador usando redirección:

```
./analyzer < data/logs_A.txt
```

4. Funcionamiento del programa

El programa utiliza llamadas al sistema de Linux ARM64:

Syscalls utilizadas:

SYS_read = 63
SYS_write = 64
SYS_exit = 93

Proceso de lectura

El programa lee bloques de datos desde la entrada estándar:

```
mov x0, #0
mov x8, #SYS_read
svc #0
```

Cada línea del archivo contiene un código HTTP.

Conversión numérica

Cada carácter leído se convierte de ASCII a entero:

```
sub w26, w26, #'0'
```

Luego se construye el número:

```
mul x23, x23, x27
add x23, x23, x26
```

Ejemplo:

- `'2'` → 2
- `'0'` → 20
- `'0'` → 200

Clasificación de códigos

Se comparan los códigos en rangos:

Códigos exitosos

cmp x23, #200
cmp x23, #299

Errores cliente

cmp x23, #400
cmp x23, #499

Errores servidor

cmp x23, #500
cmp x23, #599

5. Compilación

La compilación se realiza mediante:

make

Esto ejecuta:

```
as -o analyzer.o src/analyzer.s
ld -pie -o analyzer analyzer.o
```

El parámetro `-pie` es necesario en **Termux/Android**, ya que sin él aparece el error:

has unexpected e_type: 2

6. Ejecución

La ejecución se realiza redirigiendo el archivo de datos:

```
./analyzer < data/logs_A.txt
```

7. Resultados esperados

Dado este archivo:

```
200
201
204
404
418
500
503
200
302
401
502
```

La salida esperada es:

=== Mini Cloud Log Analyzer ===

Total de codigos: 11
Exitos 2xx: 4
Errores 4xx: 3
Errores 5xx: 3

8. Problemas encontrados

Durante la implementación se presentaron varios problemas:

1. Error de formato ELF

Al ejecutar en Termux se obtuvo:

has unexpected e_type: 2

Causa: el enlazador generaba un ejecutable `ET_EXEC`.

Solución: usar:

ld -pie -o analyzer analyzer.o

2. Fallos en impresión de resultados

El programa imprimía únicamente el título y no los conteos.

Causa: errores en el manejo de registros durante las llamadas a funciones y problemas al convertir números a texto.

Solución aplicada:

- Revisar la rutina `print_num`
- Ajustar la preservación de registros
- Verificar la lectura de datos desde `stdin`

9. Aprendizajes obtenidos

Esta práctica permitió comprender:

- Cómo trabajar con syscalls en ensamblador ARM64
- Cómo procesar datos desde buffers
- Cómo convertir texto a números
- Cómo automatizar compilación con Makefile
- Diferencias entre Linux ARM64 y Termux Android

10. Conclusión

El proyecto **Mini Cloud Log Analyzer** permitió aplicar conceptos fundamentales de bajo nivel en ensamblador ARM64, simulando el análisis de registros de un sistema.

A pesar de los errores encontrados durante la implementación, el proyecto ayudó a comprender el manejo manual de entrada, procesamiento y salida de datos sin depender de bibliotecas externas.

Es una práctica útil para fortalecer la comprensión de:

- arquitectura ARM64
- syscalls
- manejo de memoria
- depuración en entornos Linux y Termux